



Priority Queues (Heaps): The LeetCode Mastery Plan

The Golden Rule: Sorting vs. Heaps

- **Use Sorting** (`array.sort()`) when you need to order data exactly ONCE and the values **never change**. (Time: $\mathcal{O}(N \log N)$)
- **Use a Heap** (`heapq`) when the data is DYNAMIC. If you are picking a maximum/minimum, modifying it, and putting it back, or if new options are constantly unlocking, use a heap. (Time: $\mathcal{O}(\log N)$ per update).

Level 1: The Warm-Up (Max-Heap Logic)

LeetCode 1046: Last Stone Weight

- **The Goal:** Smash the two heaviest stones together. If they aren't equal, the remaining weight goes back into the pile.
- **Why do it:** Python only has a Min-Heap. This teaches you to multiply by `-1` to create a fake Max-Heap.

Python Solution

```
import heapq

class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        n = len(stones)
        if n <= 1:
            return stones[0]

        # 1. Invert all weights to create a Max-Heap
        for x in range(n):
            stones[x] = -stones[x]

        # 2. Turn the list into a heap (in-place)
        heapq.heapify(stones)

        # 3. Smash stones until 1 or 0 are left
        while len(stones) > 1:
            # Genius trick: pop biggest, pop second biggest, push the difference
            # Because they are negative, pop1 - pop2 works perfectly.
            heapq.heappush(stones, heapq.heappop(stones) - heapq.heappop(stones))

        return -stones[0]
```

Level 2: The Twist (Storing Tuples)

LeetCode 347: Top K Frequent Elements

- **The Goal:** Find the K most frequent numbers in an array.
- **Why do it:** You must learn how to put multiple pieces of data into a heap at once. A heap sorts by the *first* item in a tuple.

Python Solution

```
import heapq
from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # 1. Count frequencies
        x = Counter(nums)

        heap = []
        # 2. Push (-frequency, number) to create a Max-Heap based on frequency
        for u, v in x.items():
            heapq.heappush(heap, (-v, u))

        l = []
        i = 1
        # 3. Pop K times
        while i <= k:
            # We pop the tuple (-freq, num) and append just the num (index 1)
            l.append(heapq.heappop(heap)[1])
            i += 1

        return l
```

Level 3: The Infosys-Level Boss

LeetCode 502: IPO (Maximize Capital)

- **The Goal:** You have K projects you can do, and initial capital W . Each project has a required capital to start, and a profit. You want to maximize W .
- **The Logic (Two Data Structures!):**
 1. **Sorting:** You can only start a project if you can afford it. So, you pair up (capital, profit) and **sort** them by capital. This is static; the required capital never changes.
 2. **Heap:** As you finish projects, your capital W goes up. This *unlocks* new projects. You need a **Max-Heap** to store the profits of all the projects you can currently afford, so you can always dynamically grab the most profitable one!

The Blueprint Strategy

```
import heapq

class Solution:
    def findMaximizedCapital(self, k: int, w: int, profits: List[int], capital: List[int]) -> int:
        # Step 1: Zip and Sort!
        # Pair up the capital and profits, then sort them so the cheapest projects
        projects = sorted(zip(capital, profits))

        max_heap = []
        i = 0
        n = len(projects)

        # Step 2: We can do a maximum of K projects
        for _ in range(k):
```

```
# Step 3: UNLOCK PROJECTS
# While the next cheapest project is affordable with our current ca
while i < n and projects[i][0] <= w:
    # Push its profit into the Max-Heap (remember to make it negati
    heapq.heappush(max_heap, -projects[i][1])
    i += 1 # Move to the next project

# If our heap is empty, it means we can't afford ANY new projects r
if not max_heap:
    break

# Step 4: CASH IN
# Pop the most profitable project from the heap and add its profit
w += -heapq.heappop(max_heap)

return w
```